Lab Report 7

Digital Circuit Design

Submitted By: SYEDA AFIA SHAH

Registration No: 2115

Section: B

"On my honor , as student of University of Engineering and Technology, I have neither given nor received unauthorized assistance on this academic work"

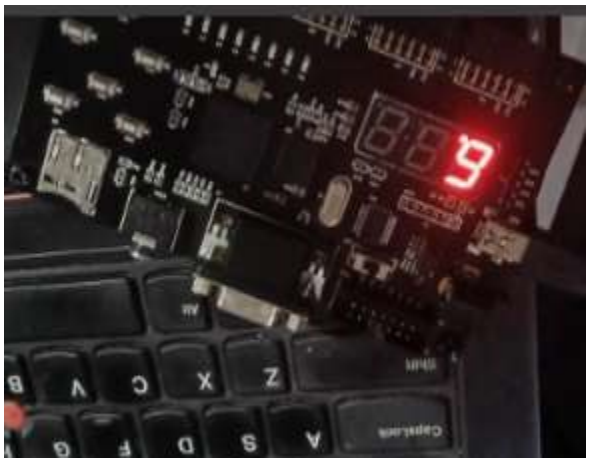
Department of Computer Systems Engineering

University of Engineering and Technology Peshawar

1-Implement 3 bit Up Counter using FPGA's clock source and clock divider

```
module clock_divider(
   input clk_in,        // 100 MHz clock input
   output reg clk_out    // 1 Hz clock output
);
   reg [26:0] counter = 0; // Enough bits to count up to 100 million

   always @(posedge clk_in) begin
      if (counter == 49_999_999) begin // 100 MHz / 2 = 50 million
         counter <= 0;
         clk_out <= ~clk_out;
      end
      else
         counter <= counter + 1;
   end
endmodule

module up_counter(
   input clk,       // 1Hz clock
   input rst,       // Reset button
   output reg [2:0] count
);
   always @(posedge clk or posedge rst) begin
      if (rst)
         count <= 0;
      else
         count <= count + 1;
   end
endmodule
module seven_segment_display(
   input [3:0] num,
   output reg [6:0] seg
);
   always @(*) begin
      case(num)
         4'd0: seg = 7'b1000000;
         4'd1: seg = 7'b1111001;
         4'd2: seg = 7'b0100100;
         4'd3: seg = 7'b0110000;
         4'd4: seg = 7'b0011001;
```

```
            4'd5: seg = 7'b0010010;
            4'd6: seg = 7'b0000010;
            4'd7: seg = 7'b1111000;
            4'd8: seg = 7'b0000000;
            4'd9: seg = 7'b0010000;
            default: seg = 7'b1111111; // Blank
        endcase
    end
endmodule
module top_module(
    input clk_100mhz,
    input rst,
    output [6:0] seg
);
    wire clk_1hz;
    wire [2:0] count;

    clock_divider u1 (.clk_in(clk_100mhz), .clk_out(clk_1hz));
    up_counter u2 (.clk(clk_1hz), .rst(rst), .count(count));
    seven_segment_display u3 (.num({1'b0, count}), .seg(seg)); // extend to 4-bit for display
endmodule
```



2-Implement 3-bit Down Counter using FPGA's clock source and clock divider

```
module clock_divider(
    input clk_in,         // 100 MHz clock input
    output reg clk_out    // 1 Hz clock output
);
    reg [26:0] counter = 0; // Enough bits to count up to 100 million

    always @(posedge clk_in) begin
```

```
        if (counter == 49_999_999) begin // 100 MHz / 2 = 50 million
            counter <= 0;
            clk_out <= ~clk_out;
        end
        else
            counter <= counter + 1;
    end
endmodule

module down_counter(
    input clk,       // 1Hz clock
    input rst,       // Reset button
    output reg [2:0] count
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            count <= 7;  // Start from 7
        else
            count <= count - 1;
    end
endmodule
module seven_segment_display(
    input [3:0] num,
    output reg [6:0] seg
);
    always @(*) begin
        case(num)
            4'd0: seg = 7'b1000000;
            4'd1: seg = 7'b1111001;
            4'd2: seg = 7'b0100100;
            4'd3: seg = 7'b0110000;
            4'd4: seg = 7'b0011001;
            4'd5: seg = 7'b0010010;
            4'd6: seg = 7'b0000010;
            4'd7: seg = 7'b1111000;
            4'd8: seg = 7'b0000000;
            4'd9: seg = 7'b0010000;
            default: seg = 7'b1111111; // Blank
        endcase
    end
```

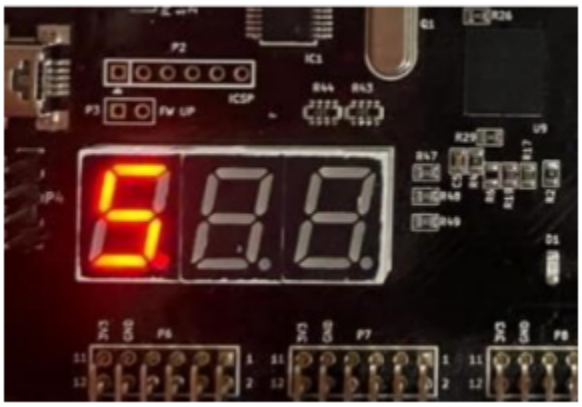
```
endmodule
module top_module(
   input clk_100mhz,
   input rst,
   output [6:0] seg
);
   wire clk_1hz;
   wire [2:0] count;

   clock_divider u1 (.clk_in(clk_100mhz), .clk_out(clk_1hz));
   down_counter u2 (.clk(clk_1hz), .rst(rst), .count(count));
   seven_segment_display u3 (.num({1'b0, count}), .seg(seg)); // extend to 4-bit for display
endmodule
```

3- Implement 3-bit Up and Down Counter using RST if the RST is high it should Count Up if it is low it should Count Down and display the count in the seven segment display.

```
module clock_divider(
   input clk_in,        // 100 MHz clock input
   output reg clk_out    // 1 Hz clock output
);
   reg [26:0] counter = 0; // Enough bits to count up to 100 million

   always @(posedge clk_in) begin
      if (counter == 49_999_999) begin // 100 MHz / 2 = 50 million
         counter <= 0;
         clk_out <= ~clk_out;
      end
      else
         counter <= counter + 1;
   end
```

```
endmodule

module up_down_counter(
    input clk,        // 1Hz clock
    input rst,        // Up/Down control: 1 for Up, 0 for Down
    output reg [2:0] count
);
    always @(posedge clk) begin
        if (rst)
            count <= count + 1;  // Count Up
        else
            count <= count - 1;  // Count Down
    end
endmodule
module seven_segment_display(
    input [3:0] num,
    output reg [6:0] seg
);
    always @(*) begin
        case(num)
            4'd0: seg = 7'b1000000;
            4'd1: seg = 7'b1111001;
            4'd2: seg = 7'b0100100;
            4'd3: seg = 7'b0110000;
            4'd4: seg = 7'b0011001;
            4'd5: seg = 7'b0010010;
            4'd6: seg = 7'b0000010;
            4'd7: seg = 7'b1111000;
            4'd8: seg = 7'b0000000;
            4'd9: seg = 7'b0010000;
            default: seg = 7'b1111111; // Blank
        endcase
    end
endmodule
module top_module(
    input clk_100mhz,   // 100MHz clock from FPGA
    input rst,          // RST acts as up/down control: 1 = UP, 0 = DOWN
    output [6:0] seg,   // 7-segment display segments
    output [3:0] an     // 7-segment display enable (anodes)
);
```

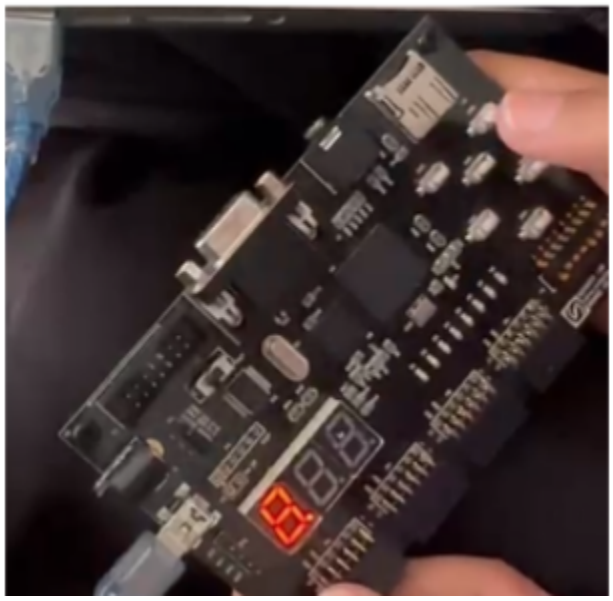
```
    wire clk_1hz;
    wire [2:0] count;

    // Instantiate Clock Divider (100MHz -> 1Hz)
    clock_divider u1 (
        .clk_in(clk_100mhz),
        .clk_out(clk_1hz)
    );

    // Instantiate 3-bit Up/Down Counter
    up_down_counter u2 (
        .clk(clk_1hz),
        .rst(rst),
        .count(count)
    );

    // Instantiate 7-Segment Display Decoder
    seven_segment_display u3 (
        .num({1'b0, count}), // Extend 3-bit to 4-bit
        .seg(seg)
    );

    // Activate only 1 display (assuming 4 digits, active low)
    assign an = 4'b1110; // Enable only 1st display (modify if needed)

endmodule
```



4-Implement Lab 6 using FPGA's clock source.

```
module clock_divider(
    input clk_in,
    output reg clk_out
```

```
);
    reg [26:0] counter = 0;

    always @(posedge clk_in) begin
        if (counter == 49_999_999) begin
            counter <= 0;
            clk_out <= ~clk_out;  // Toggle every 0.5 sec → 1Hz output
        end
        else
            counter <= counter + 1;
    end
endmodule
module ring_counter(
    input clk,
    input rst,
    output reg [7:0] out
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            out <= 8'b0000_0001; // Start with bit 0 ON
        else
            out <= {out[6:0], out[7]}; // Rotate left
    end
endmodule

module seven_segment_ring(
    input [2:0] num,  // 3-bit position input
    output reg [6:0] seg
);
    always @(*) begin
        case(num)
            3'd0: seg = 7'b1000000; // 0
            3'd1: seg = 7'b1111001; // 1
            3'd2: seg = 7'b0100100; // 2
            3'd3: seg = 7'b0110000; // 3
            3'd4: seg = 7'b0011001; // 4
            3'd5: seg = 7'b0010010; // 5
            3'd6: seg = 7'b0000010; // 6
            3'd7: seg = 7'b1111000; // 7
            default: seg = 7'b1111111; // Blank
```

```
        endcase
    end
endmodule

module top_module(
    input clk_100mhz,  // 100MHz FPGA clock
    input rst,         // Reset button
    output [6:0] seg,  // 7-segment segments (abcdefg)
    output [3:0] an    // 7-segment enable (active low)
);
    wire clk_1hz;
    wire [7:0] out;
    wire [2:0] position;

    // Clock Divider
    clock_divider u1 (
        .clk_in(clk_100mhz),
        .clk_out(clk_1hz)
    );

    // 8-bit Ring Counter
    ring_counter u2 (
        .clk(clk_1hz),
        .rst(rst),
        .out(out)
    );

    // Find which bit is ON (position 0-7)
    assign position = (out[0]) ? 3'd0 :
                      (out[1]) ? 3'd1 :
                      (out[2]) ? 3'd2 :
                      (out[3]) ? 3'd3 :
                      (out[4]) ? 3'd4 :
                      (out[5]) ? 3'd5 :
                      (out[6]) ? 3'd6 :
                      (out[7]) ? 3'd7 : 3'd0; // Default

    // 7-segment Display
    seven_segment_ring u3 (
        .num(position),
```

```
        .seg(seg)
    );

    // Activate only one display (first digit)
    assign an = 4'b1110; // Active low, only one ON

endmodule
```